

Enterprise RAG + Agent System — Full Roadmap

A production-grade RAG assistant with agentic reasoning, enterprise controls, and MLOps observability. Original 8-phase plan, with security, testing, and reliability gaps folded in where they naturally belong.

Standing practice, not a phase: start sanity-checking retrieval quality as soon as retrieval exists in Phase 1 — a handful of manual "does this pull the right chunk" checks, even before Phase 4's formal golden dataset and eval dashboard exist. The full eval infrastructure stays in Phase 4 because it needs Phase 3's data model to log against properly, but the *habit* of testing before improving should start on day one. Build → test → improve → test again, not build everything → evaluate once at the end.

Code architecture, not a phase: build this in from Phase 1, since it costs nothing extra if done from the start and a lot to retrofit later.

- **Dependency injection / service layer** — `LLMService`, `RetrieverService`, `AgentService`, `EmbeddingService` instead of instantiating clients inline everywhere. This is what makes your test suite in Phase 6 actually mockable instead of hitting real APIs.
- **Repository pattern** — `FastAPI → Service layer → Repository → Database`, not raw SQL in your route handlers. Example: `ChatService → ChatRepository → PostgreSQL`. Makes swapping databases or mocking data in tests far easier, and it's the same shape production systems actually use.

Phase 1: Enterprise RAG (Week 1-2)

Goal: Build a production-ready RAG assistant.

- **[Expanded] Document ingestion pipeline** — not just Upload → Chunk → Embed:
`Upload → Validate (file type/size) → Virus scan → OCR (if scanned PDF) → Extract metadata → Chunk → PII detection/redaction → Embedding → Store`
- **[Added] File validation at upload** — reject anything outside an allowed file-type list (e.g. PDF/DOCX only) and enforce a max file size before the file touches virus scan or OCR. This is the cheapest, most common bad-input case, and catching it at the door is far cheaper than catching it downstream.
- **[Added] Metadata on ingestion** — filename, author, upload date, department, tags, document version. This is what makes filtered retrieval possible later, so capture it from week 1 even if you don't use it yet.
- **[Moved from Phase 7] Hybrid search** — BM25 + vector search combined, not vector-only. This is close to default in modern production RAG now, not an enhancement; a keyword miss (acronyms, exact policy numbers, names) is a common vector-search failure mode that BM25 catches. (Reranking on top stays in Phase 7 — hybrid search

alone already gives most of the quality jump, and bundling both into week 1-2 overloads the first milestone.)

- LLM answers — **[expanded]** build the system prompt as a Jinja template with variables (`(role)`, `(retrieved_context)`, `(conversation_history)`, `(user_question)`, `(instructions)`), not a raw string. Trivial now, painful to refactor once you have prompt versioning in Phase 4.
 - Source citations
 - Chat history
 - **[Moved from Phase 7] Conversation memory** — an agent without memory of the last turn can't resolve "summarize this" → "translate it to Hindi." This is core to feeling like an agent, not a nice-to-have; build it alongside chat history rather than bolting it on in week 7.
 - Streaming responses
 - Multi-document search
 - **[Added] Retrieval debug mode** — a toggle (admin/dev only) that shows the raw retrieved chunks, their similarity scores, and metadata for any query. This will save you hours once retrieval quality issues show up, and it's nearly free to add while you're already building retrieval.
 - **[Added] Configuration management from day one** — `(chunk_size)`, `(top_k)`, `(temperature)`, `(embedding_model)`, `(llm_model)`, `(reranker)`, `(cache_ttl)` all live in a YAML/env config, never hardcoded. Cheap to do early, painful to retrofit later.
 - **[Added] Async ingestion** — upload returns immediately, embedding happens on a background queue (this is your Phase 6 Celery worker — just wire the upload endpoint to it from the start instead of doing it synchronously first and refactoring later)
-

Phase 2: Agent System (Week 3)

Instead of always searching documents, introduce an agent.

Tools

- Vector Search — **[expanded]** support metadata-filtered search: "only HR documents," "only 2026 documents," "only version 3," "only docs uploaded by Finance." This is one of the highest-value enterprise features on this list — it's what makes the vector search tool feel like real enterprise search instead of a demo.
- SQL Database
- Web Search (optional)
- Calculator
- CSV Analysis

[Added] Agent safety controls

- Max-iteration cap on the agent loop (e.g. 5 steps) — prevents infinite tool-calling loops
 - Read-only DB credentials for the SQL tool — no `DROP`/`DELETE`/`UPDATE` permitted
 - Sanitize retrieved chunk content before inserting into the prompt (basic prompt-injection defense against poisoned documents)
 - **[Added] SQL tool input validation** — validate/parameterize whatever the agent hands to the SQL tool before it runs, on top of the read-only credentials. Read-only creds stop damage; input validation stops a malformed or injected query from running at all.
-

Phase 3: Enterprise Features (Week 4)

This is what separates a demo from a production application.

Authentication

- JWT
- Login
- Signup

Authorization

- Admin / HR / Employee roles
- **[Added] Document-level access control** — tag chunks with access metadata at ingestion time; filter retrieval results by the requesting user's permissions *before* they reach the LLM (an Employee query must never surface HR-only chunks just because they're semantically similar)

User Management

- Chat history
- Multiple users
- Session management

Document Management

- Upload
- Delete
- Re-index
- Versioning

[Added] Secrets management

- Move API keys, DB credentials, JWT secrets out of `.env` into AWS Secrets Manager or Vault (at least for the deployed version)

[Added] Audit logs — separate from normal application logs

- Who uploaded/deleted a document
- Who queried confidential/restricted docs
- Who logged in, failed login attempts
- This is a distinct table from your debug/error logs — it's a compliance record, so it should be append-only and never overwritten

[Added] API versioning

- `/v1/chat`, `/v1/upload` from the start, so a future `/v2/chat` doesn't require a breaking migration. Trivial to set up now, awkward to retrofit.

[Scaled down] Admin dashboard basics

- Full analytics dashboard stays in Phase 7, but the *admin actions* belong here: disable a user, re-index a document, approve an uploaded document before it's searchable. Keep this to 3-4 actions — a full ops console is its own project.

[Added] Input validation — file validation (Phase 1) and SQL tool validation (Phase 2) handle the cheapest, earliest cases; this is where it becomes systematic and enforced across every route:

- FastAPI/Pydantic request schema validation on every endpoint, not just the obvious ones
 - Prompt length validation — cap it, and tie the cap into the per-user token budget work in Phase 4
 - Consistent invalid-request handling with proper HTTP status codes (422 for validation errors, not a generic 500) instead of ad hoc error shapes per route
-

Phase 4: AI Observability & MLOps (Week 5)

Track

- LLM latency
- Prompt tokens / completion tokens
- Cost per request
- Retrieval latency
- Vector search latency
- Number of retrieved chunks
- User feedback (🍌 🗨)

Evaluation

- Golden dataset
- Faithfulness
- Answer relevancy
- Context precision
- Hallucination rate

Logging

- Prompt
- Retrieved chunks
- Response
- Errors

[Added] Cost and reliability controls

- Per-user / per-org token budget or spend cap
- Fallback LLM provider if primary API is down or rate-limited

[Added] Prompt versioning — don't store prompts as raw strings scattered in code. Keep a `prompts` table with `version`, `text`, `created_at`, and log which prompt version answered every request. This is genuinely underrated — real AI teams do this specifically so they can roll back a bad prompt change.

[Added, scaled down] Lightweight model registry — you don't need a full MLflow-scale system. A simple table logging `embedding_model`, `llm_model_version`, `prompt_version`, `retriever_version` per request is enough to let you compare "BGE-large-v1.5 vs BGE-M3" performance later without guessing which config produced which answer.

[Added] Search quality metrics, separate from answer quality — measure retrieval *before* generation:

- Recall@5, Recall@10
- MRR (mean reciprocal rank)
- Hit rate This matters because a bad answer is often actually a retrieval problem, not a generation problem — these metrics let you tell the difference.

[Added] Evaluation dashboard — surface faithfulness, answer relevancy, context precision, latency, cost, and top failing queries in one view. Doubles as your best demo screen.

Phase 5: Deployment (Week 6)

Use

- Docker / Docker Compose
- Kubernetes
- GitHub Actions
- Terraform
- AWS

Services

- FastAPI
- PostgreSQL
- Redis
- Qdrant
- Prometheus
- Grafana

[Added] CI/CD refinement

- Staging environment before prod
- Basic canary or blue-green deploy step in GitHub Actions

[Added] Basic monitoring & alerting — Prometheus/Grafana above gets you metrics; this is what turns "CPU is at 90%" into "🚨 CPU has been above 90% for 5 minutes." Keep it lean for a solo project — Grafana's built-in alerting covers this without standing up a separate Alertmanager stack:

- Grafana Alerts (built-in, no separate Alertmanager needed at this scale)
- Alert rules: high API latency, high error rate, LLM provider unavailable, high token usage/spend, disk/memory threshold exceeded
- Single notification channel (email or Slack webhook) — no on-call rotation or escalation policy needed here
- **[Optional stretch]** Full Prometheus Alertmanager with multi-channel routing and escalation — treat this the same way as OpenTelemetry below: genuinely useful, but its own project, not core to shipping.

[Optional stretch] OpenTelemetry — Prometheus + Grafana already covers metrics; OpenTelemetry adds distributed *tracing*, so you can see one request's full path (User → FastAPI → Retriever → Qdrant → LLM → Response) with per-hop timing. Genuinely professional, but treat it as a stretch add after the core stack works — don't let it block the deploy milestone.

Phase 6: Production Improvements (Week 7)

Redis Cache — embeddings, LLM responses, retrieval results

- **[Added] Semantic caching** — reuse cached answers for queries similar enough by embedding distance, not just exact-match keys

Background Workers (Celery or similar)

- Generate embeddings
- Process uploads
- Re-index documents
- Rate limiting
- **[Expanded] Retry and fallback strategy** — spell this out as an actual flow, not just "retry logic": LLM timeout → retry → fallback model → return graceful error → log incident. The graceful-error step matters as much as the retry — never let a failure surface as a raw stack trace to the user.
- Structured logging
- **[Expanded] Health endpoints** — /health, /ready, /live as separate endpoints, not one generic /health. Small touch, but it's the convention Kubernetes deployments expect (liveness vs readiness probes check different things).
- Metrics

[Added] Testing (*new addition to this phase*)

- Unit tests: chunking, retrieval, tool logic
- Integration tests: full RAG pipeline end-to-end
- Load testing (Locust or k6) — see how the system holds up under concurrent users, pairs naturally with rate limiting work above

[Added] Data lifecycle

- Retention policy for chat history and logs
 - Confirm PII redaction (from Phase 1) is enforced consistently at scale
-

Phase 7: Nice-to-Have AI Features (Week 8)

- Hybrid Search (BM25 + Vector Search) — **[moved to Phase 1]**, see above
- Reranking (Top 20 → Reranker → Top 5 → LLM)
- Conversation Memory — **[moved to Phase 1]**, see above
- Suggested/Related Questions after each answer
- Feedback System (👍 Helpful / 👎 Not Helpful)
- Analytics Dashboard — users, questions/day, most searched documents, average latency, token usage, estimated cost

- **[Added] Failure-focused analytics** — top failed searches, top unanswered questions. These matter more than "most searched documents" because they tell you where your knowledge base is actually weak, not just what's popular.
-

Phase 8: Documentation

- **[Expanded] Three separate architecture diagrams, not one:**
 - *Logical architecture* — `User → Frontend → FastAPI → Agent → Tools → LLM`
 - *Deployment architecture* — `AWS → ALB → Kubernetes → Pods → Qdrant / Redis / Postgres`
 - *Data flow* — `Upload → Chunk → Embedding → Qdrant → Retriever → LLM → Answer`These answer different questions (how it's structured vs. where it runs vs. how data moves) and reviewers will look for exactly this separation.
 - Sequence diagrams
 - ER diagram
 - Database schema
 - API documentation (Swagger) — **[expanded]** also expose the raw OpenAPI JSON automatically (FastAPI gives you this for free at `/openapi.json`) so the API is machine-consumable, not just human-browsable
 - Deployment guide
 - Setup instructions
 - Performance benchmarks
 - Design decisions
 - Future improvements — **[note]** this is the natural place to mention multi-tenancy (Phase 9 below) as a planned-but-not-built extension
 - **[Added] Threat model summary** — one page: what document-level access control prevents, what the prompt-injection sanitization covers, what it doesn't (be honest about the limits — this is what a senior reviewer will probe first)
-

Phase 9 (optional stretch, not part of core 8-week plan): Multi-Tenant Architecture

Isolating tenants properly — separate vector collections per company, tenant-aware auth on every single query path, fully isolated data — is a substantial project on its own, not a bolt-on. It's the single most "enterprise SaaS" feature on any wishlist, but attempting it inside an already-packed 8-week solo build is the most common way these projects stall out half-finished.

Recommendation: don't build this into the core roadmap. If you finish everything else with time to spare, this is the natural next thing to add — but treat it as a v2 project, not a Phase 9

you're expected to ship alongside the rest.

Phase 10: Portfolio Deliverables

Execution and presentation, not new features. This is the difference between a project that demonstrates its own value and one that requires you to explain it.

- Live deployed demo
 - GitHub repository, clean and organized
 - Excellent README (setup instructions, architecture overview, screenshots)
 - Architecture diagrams (already produced in Phase 8 — link them here)
 - 5-10 minute demo video
 - System design document (Phase 8, linked)
 - API collection (Postman/Bruno) for anyone who wants to try the endpoints directly
 - Benchmark report (latency, cost, retrieval quality numbers — you already have all of this from Phase 4)
 - Screenshots
 - Resume bullet points, written now while the details are fresh
-

Final verdict: freeze the roadmap here

This document is complete. The highest-value use of remaining time from this point is building, testing, and polishing — not adding more ideas. Target execution quality over Phase 6, not new scope:

- 90%+ test coverage on critical modules (retrieval, agent routing, auth)
- Clean, modular architecture (dependency injection + repository pattern, per the note above)
- Comprehensive logging
- Well-written documentation
- Professional UI/UX
- Live cloud deployment
- A strong README with diagrams and setup instructions

A complete, polished version of this roadmap — built and finished — is worth far more than a longer roadmap that never ships.

Summary of what changed

Phase	Additions
1	Full ingestion pipeline (file type/size validation , virus scan, OCR, metadata extraction, PII redaction), metadata capture for filtering, config management from day one, async ingestion, hybrid search (moved from Phase 7) , conversation memory (moved from Phase 7) , prompt templates (Jinja), retrieval debug mode
2	Metadata-filtered vector search, agent iteration cap, read-only SQL creds, SQL tool input validation , prompt-injection sanitization
3	Document-level access control, secrets management, audit logs, API versioning, scaled-down admin actions, input validation (Pydantic schema checks, prompt length caps, consistent error codes)
4	Per-user cost caps, fallback LLM provider, prompt versioning, lightweight model registry, search quality metrics (Recall@k/MRR), evaluation dashboard
5	Staging environment, canary/blue-green deploy, basic monitoring & alerting (Grafana Alerts, single notification channel) , OpenTelemetry and full Alertmanager (optional stretch)
6	Full testing suite, semantic caching, data retention policy, expanded retry/fallback flow, <code>/health</code> <code>/ready</code> <code>/live</code> endpoints
7	Failure-focused analytics (top failed searches, top unanswered questions) — reranking stays here as the enhancement layer on top of Phase 1's hybrid search
8	Three separate architecture diagrams (logical/deployment/data flow), threat model summary, OpenAPI JSON exposure
9 (new, optional)	Multi-tenant architecture — deliberately kept out of the core 8-week plan

Standing practice (not tied to a phase): evaluate retrieval quality as soon as it exists, not just at Phase 4 — see the note at the top of this doc.

What was discarded and why

- **Full multi-tenant SaaS isolation** — moved to an explicit optional Phase 9. Real tenant isolation (separate vector collections, tenant-aware auth on every path) is a multi-week project by itself; folding it into the core 8 weeks is the most common reason these builds stall half-finished.
- **Full-scale model registry (MLflow-style)** — replaced with a lightweight version-tracking table. You get the same benefit (compare model/prompt versions later)

without building registry infrastructure that's overkill for a solo project.

- **Full admin ops console** — replaced with 3-4 essential admin actions (disable user, re-index doc, approve upload) inside Phase 3, with the analytics view staying in Phase 7 where it already belonged.
- **Reranking moved earlier alongside hybrid search** — considered, but kept in Phase 7. Hybrid search alone delivers most of the retrieval-quality gain; adding a reranker too would overload the Phase 1 milestone for a smaller marginal improvement.

Nothing else was removed. The rest slots into existing weeks rather than needing extra time — except testing in Phase 6 and the expanded ingestion pipeline in Phase 1, which may each add a few extra days if done properly.